

On training a local, small-scale chess AI

Hegyi Erik

June 29, 2026

Contents

1	How to represent the chessboard	1
1.1	What needs to be represented?	1
1.2	How should we represent the game?	1
1.3	How can we optimize the representation?	3
2	What should the model architecture look like?	4
3	How to predict the best move?	5
4	How do we train the model?	6
4.1	How can we start training the model?	6
4.2	How can the model start learning by playing against itself? . .	7
4.3	How can we improve the model?	8
4.4	How can we train a fully working chess AI?	9

1 How to represent the chessboard

1.1 What needs to be represented?

A **chessboard** is a **table**, which has **8 rows** and **8 columns**.

Each cell of this table has exactly one value - what piece is occupying that cell. This can either be:

- None
- White or black pawn
- White or black knight
- White or black bishop
- White or black rook
- White or black queen
- White or black king

This is all the information that a chessboard contains - where the pieces are - and all the information that *we* need to represent. Everything else is encoded by the rules of the game, not by the state of the board itself.

1.2 How should we represent the game?

If we were to give each piece an "*ID*", like 1 for pawn and 4 for rook, then we would quickly run into a problem - the pieces could be added together. In chess, multiplying a pawn by 4 does not result in a rook, since it is an undefined operation.

Thus, we need to find a better representation, and that is going to be through **channels**.

Instead of representing the board as one **matrix**, we are going to represent it as a **tensor**.

Let P be the number of pieces that exist in a game of chess. Since we are dealing with classic chess, $P = 6$. These 6 pieces have already been listed in section 1.1.

Let O be the number of players, with $O = 2$ holding true for classic chess. Let W be the width of the chessboard, and let H be the height of the chessboard. In classic chess $W = H = 8$.

We are going to deal with **binary matrices**. Each piece type is going to have a matrix of its own, which encodes where these pieces are, and where they are not.

In reality, each piece will have two of these matrices, one for encoding the white pieces of this type, and one for encoding the black pieces of this type. From now on, whenever we are talking about a piece type's dimension, we are talking about one specific colour.

For example, before the first move has been made, the binary matrix of the white pawn would look like this:

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Since the pawns are only in one row, all the 1 values are found in that row. Anywhere where a white pawn is not to be found, we place a 0.

With this method, we can no longer look for any operation which will transform a certain piece into another one, since all piece types exist in different dimensions.

Thus, our **state tensor** $\mathbf{T}$ is going to be of the shape:

$$\mathbf{T} \in \mathbb{B}^{PO \times W \times H} \tag{1.2.1}$$

1.3 How can we optimize the representation?

We can see that the state tensor only contains ones and zeros, since it is a binary tensor.

According to (1.2.1), the tensor is going to have the dimensions of:

$$12 \times 8 \times 8$$

Or in other words, the tensor is going to contain 768 **bits**. This is exactly $12 \cdot 64$, which means that, since we are dealing with bits only, we can encode it into twelve **64-bit integers**, one for each piece.

2 What should the model architecture look like?

It is important to notice that the chessboard is in the same format as a **picture** with twelve channels. It is important for our chess AI interpret data the same no matter where the pieces are, since it does not matter where the pieces are located, it only matters how they are located with respect to all the other pieces. This makes it obvious that we must use a **convolutional neural network**.

The input of the neural network is going to be a tensor of size $PO \times W \times H$. Then, we are going to use multiple convolutional layers to gradually extract information from the board, with the layers having 2x2 kernels. We are going to increase the number of channels from 12 to 16, 32, 64, 128, 256, 512, 1024 and 2048. Inbetween, we are going to use max- and average pooling to reduce the number of spatial dimensions.

We are also going to use skip connections to improve the learning efficiency of the model.

At the end, when we have a tensor of size $2048 \times 1 \times 1$, we are going to flatten the tensor into a vector, and then create a linear, fully connected layer, which reduces the dimensionality into a single scalar.

Then, we are going to use the hyperbolic tangent function to normalize the output between -1 and +1.

3 How to predict the best move?

4 How do we train the model?

4.1 How can we start training the model?

4.2 How can the model start learning by playing against itself?

4.3 How can we improve the model?

4.4 How can we train a fully working chess AI?